

11 · Seguridad

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **5e91229**

Fecha de generacion: **2026-08-27**

Generado a partir de 11-security.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Modelo de amenaza resumido
2. Autenticación y autorización
3. Gestión de secretos
4. Validación y saneado
5. Cifrado
6. Registro y auditoría
7. Exposición de información
8. CORS, CSRF y superficie web
9. Carga de archivos
10. Dependencias
11. Permisos que el producto pide
12. Autoprotección del agente
13. Controles implementados — resumen
14. Controles ausentes o no comprobados
15. Qué se probó y qué no en este análisis

Análisis de seguridad **del propio producto**, no de lo que el producto detecta. Qué superficie expone, qué controles implementa, cuáles no y qué riesgos quedan abiertos. Ningún hallazgo de este documento incluye valores explotables.

1. Modelo de amenaza resumido

RootCause es una herramienta de escritorio local, sin servidor y sin cuentas. Su superficie de ataque es, por tanto, pequeña y muy concreta:

Activo	Por qué importa	Amenaza principal
Baselines en SQLite	Definen qué se considera «normal»	Manipulación: un atacante que las reescriba consigue que su persistencia parezca conocida
Historial e incidentes	Evidencia forense	Borrado o alteración
Auditoría	Registro de acciones	Borrado
Configuración	Umbrales y listas de confianza	Manipulación: añadir un prefijo de confianza amplio silencia heurísticas
El propio binario	Es quien observa	Sustitución o finalización
Clave del proveedor IA	Credencial del usuario	Exposición en la lista de procesos

Suposición central del diseño, declarada en el código: RootCause corre en espacio de usuario y no puede defenderse de un atacante con root. `services/resilience.rs` lo dice sin adornos: *no hay supervisor de nivel sistema ni protección contra un root decidido*.

2. Autenticación y autorización

Aspecto	Estado
Sistema de cuentas propio	No existe
Sesiones	No existen
Roles y permisos internos	No existen
Tokens propios	No existen
Autorización efectiva	La del sistema operativo: UID del proceso, TCC y SIP

Esto no es una carencia sino una decisión: una herramienta local que se ejecuta con los privilegios del usuario no gana nada añadiendo una capa de autenticación propia; solo añadiría credenciales que proteger.

Lo que sí implementa es una política local de acciones, que restringe *por encima* de lo que el sistema permitiría:

Política	Implementación	Efecto
Nunca finalizar el propio proceso	<code>pid == own_pid</code>	Rechazo explícito
Nunca finalizar <code>pid <= 1</code>	<code>can_terminate_ process</code>	Rechazo
Trece procesos protegidos	Lista literal, no configurable	Rechazo aunque el usuario tenga permiso

Acciones manuales desactivables	<code>remediation.manual_actions_enabled</code>	Bloquea <code>terminate_process</code>
Sin acciones automáticas	<code>automatic_actions_enabled</code> nunca se lee	El producto nunca actúa solo
No aplicar reglas de firewall	<code>suggest_block_ip</code> devuelve texto	La ejecución es del usuario

3. Gestión de secretos

Secreto	Cómo se maneja	Valoración
Clave del proveedor IA	Se lee de una variable de entorno cuyo nombre está en la configuración	Correcto: no se persiste
Cualquier otro	No existe ninguno	—

Verificado: no hay ninguna credencial, token ni clave literal en el repositorio. La búsqueda de patrones habituales (`api_key`, `secret`, `token`, `password`, `Bearer`) solo devuelve el nombre de la variable de entorno, la cabecera de `curl` construida en tiempo de ejecución y los textos de la documentación.

Riesgo real detectado: en `ai:post_json`, la clave viaja como argumento de `curl`:

```
"-H", &format!("Authorization: Bearer {api_key}")
```

Los argumentos de un proceso son visibles para cualquier usuario del equipo mediante `ps`. El cuerpo de la petición sí se protege (va por `stdin` con `--data @-`, con un comentario que explica exactamente esa razón), pero la cabecera no. Severidad **media** y ventana muy corta —solo mientras dura la petición—, y solo aplica si el usuario activó la IA. Se registra en [15 · Riesgos](#) con su recomendación: usar `--header @-` o un archivo de configuración temporal de `curl`.

4. Validación y saneado

Entrada	Saneado	Dónde
Salidas de utilidades del sistema	Parseo tolerante con guardas; nada se ejecuta	Todos los parsers
Rutas de binarios	Se usan como argumento de <code>codesign</code> y <code>open</code> , sin construir shell	<code>macos.rs</code>
Título y mensaje de notificación	Las comillas dobles se sustituyen por simples antes de componer el AppleScript	<code>macos::notify</code>
Texto de reporte Markdown	<code>escape</code> neutraliza <code>\ </code> y saltos de línea	<code>report.rs</code>
Nombres de tabla en SQL	Interpolados, pero solo desde literales del módulo	<code>persistence.rs::trim_table</code>
Resto de valores SQL	Parámetros ligados (<code>params![...]</code>)	Todo <code>persistence.rs</code>
Configuración JSON	<code>serde</code> con valores por defecto y advertencia	<code>config.rs</code>
IP para <code>block-ip</code>	<code>network::extract_ip</code> ; si no hay IP válida, error	<code>inspector.rs</code>

4.1 Inyección de comandos

No hay ejecución de shell en ninguna parte del producto. Todas las llamadas usan `std::process::Command` con el binario y sus argumentos separados; no se construye ninguna cadena de comando ni se invoca `sh -c`. Eso elimina de raíz la inyección de

comandos, incluso con rutas de archivo que contengan espacios, comillas o ;.

La única construcción de código en tiempo de ejecución es el AppleScript de `notify`, y ahí el saneado es parcial: se sustituyen comillas dobles, pero no barras invertidas. Un título de alerta con `\` podría alterar el script. **Los títulos y detalles de alerta los genera el propio producto**, no el usuario ni un tercero, así que la explotabilidad es baja; aun así se anota como hallazgo menor.

4.2 Inyección SQL

Todas las consultas con datos variables usan parámetros ligados. La única interpolación es el nombre de tabla en `trim_table`, alimentado con dos literales del propio módulo ("`snapshots`" e "`incidents`"), lo que el código documenta explícitamente. **No hay inyección SQL posible desde datos externos.**

4.3 Recorrido de rutas

`expand_home` y las rutas de escaneo se construyen a partir de constantes del código y de `dirs::home_dir()`; no se aceptan rutas de entrada del usuario salvo en dos lugares:

Punto	Entrada	Riesgo
<code>snapshot -- output <ruta></code>	Ruta de escritura	El proceso escribe donde el usuario indique, con sus propios permisos. Comportamiento esperado de un CLI
<code>reveal_in_finder(path)</code>	Ruta a revelar	Procede de una entrada de persistencia leída del sistema, no de entrada libre

5. Cifrado

Aspecto	Estado
Cifrado en reposo del SQLite	No. Depende de FileVault
Cifrado de la configuración	No. Es JSON en claro
Cifrado en tránsito	Solo la petición de IA, por HTTPS si el endpoint lo es
Verificación del certificado	La de <code>curl</code> por defecto (no se usa <code>-k</code> ni <code>--insecure</code>)
Firma de los artefactos publicados	No: los binarios no están firmados ni notarizados
Hashes de integridad	Sí: <code>SHA256SUMS.txt</code> en cada release

NO DOCUMENTADO EN EL REPOSITORIO: no se comprueba que `ai.endpoint` sea `https://`. Un endpoint `http://` enviaría el incidente y la clave en claro. Se registra como riesgo.

6. Registro y auditoría

Qué se audita	Detalle
Trece tipos de acción	Ver 07 · Base de datos §3.3
Éxitos y fallos	<code>success</code> booleano y <code>detail</code> con el mensaje o el error
Eventos del agente	Cierre abrupto, cambio de configuración, cierre limpio
Consulta	<code>rootcause audit [N]</code> o la sección Historial

Limitaciones de la auditoría, honestas:

- Vive en el mismo archivo SQLite que todo lo demás, en el directorio del usuario: **quien puede escribir ahí puede alterarla**. No hay firma, ni cadena de hashes, ni copia externa.
- No se recorta, así que crece indefinidamente; eso es bueno para la evidencia y conviene saberlo para el tamaño del archivo.
- No hay integración con el log unificado de macOS ni con un SIEM.

7. Exposición de información

Vector	Estado
Puertos a la escucha	Ninguno : el producto no abre sockets de escucha
Salida de red	Solo la IA opcional, apagada por defecto
Telemetría	Ninguna
Datos en la línea de comandos	La clave de IA, mientras dura la petición (§3)
Permisos de los archivos creados	Los que resulten de la umask del usuario: NO DOCUMENTADO EN EL REPOSITORIO ; no se fijan explícitamente
Datos en los exportes	La captura completa, incluidas rutas de usuario; van a ~/Downloads
Datos en el reporte	Nombre del equipo, modelo, rutas, IP; el usuario decide si lo comparte

El exporte y el reporte son evidencia forense y contienen información identificable del equipo. El producto no los anonimiza ni ofrece un modo redactado: **NO DOCUMENTADO EN EL REPOSITORIO**. Quien comparta un reporte debe revisarlo antes.

8. CORS, CSRF y superficie web

No aplican. El producto no expone HTTP, no tiene navegador embebido, no renderiza HTML y no acepta peticiones entrantes de ningún tipo. La landing (**landing/**) es una página estática sin formularios ni JavaScript de red que se publica en GitHub Pages.

9. Carga de archivos

No aplica. El producto no recibe archivos. Lee archivos del sistema (plists, **TCC.db**, cachés) y escribe archivos propios.

10. Dependencias

10.1 Directas

Crate	Versión	Superficie que aporta	Nota
anyhow	1	Manejo de errores	Sin superficie externa
chrono	0.4	Fechas	—
dirs	5	Rutas del usuario	—
eframe / egui	0.29	Interfaz gráfica, OpenGL vía glow	La mayor superficie del árbol; opcional
plist	1	Parseo de plists de terceros	Entrada no confiable: procesa archivos que un atacante podría escribir

<code>rusqlite</code>	0.32 (bundled)	SQLite embebido	Abre bases ajenas (TCC.db) en solo lectura
<code>serde / serde_json</code>	1	Serialización	Procesa la configuración local
<code>sysinfo</code>	0.32	Métricas del sistema	—
<code>walkdir</code>	2.5	Recorrido de directorios	Sin seguir enlaces simbólicos

Total con transitivas: **326 paquetes** en `Cargo.lock`.

Los dos crates que procesan **entrada potencialmente hostil** son `plist` y `rusqlite`: un `plist` malformado o una `TCC.db` corrupta llegan al parser. En ambos casos el código maneja el error (`ok()` y `Result`) y sigue, de modo que un fallo de parseo degrada la superficie pero no tumba el proceso. Un fallo de memoria dentro del crate sería otra historia; ambos son crates ampliamente usados y escritos en Rust seguro (`rusqlite` envuelve C).

10.2 Comprobación de vulnerabilidades

REQUIERE VALIDACIÓN: no se ejecutó `cargo audit` en este análisis y el repositorio no incluye ese paso en la CI. Recomendación en [15 · Riesgos](#).

10.3 Cadena de suministro de la CI

Elemento	Fijación	Riesgo
<code>actions/checkout@v7, upload-artifact@v7, configure-pages@v6, upload-pages-artifact@v5, deploy-pages@v5</code>	Por etiqueta mayor	Estándar; una etiqueta puede moverse
<code>dtolnay/rust-toolchain@stable</code>	Por rama	Idem
<code>Swatinem/rust-cache@v2</code>	Por etiqueta mayor	Idem
<code>softprops/action-gh-release@v3</code>	Por etiqueta mayor	Tiene <code>contents: write</code>
<code>npx --yes markdownlint-cli2</code>	Sin versión	Descarga la última versión en cada ejecución

Fijar las acciones por SHA y `markdownlint-cli2` por versión reduciría esta superficie. Se registra como riesgo bajo, porque ninguno de esos pasos toca el binario que se publica salvo `action-gh-release`.

11. Permisos que el producto pide

Permiso	Cuándo	Alcance real	Se puede negar
Acceso total al disco	Lo concede el usuario en Ajustes; no hay diálogo automático	Leer <code>TCC.db</code> ; en la práctica, leer cualquier archivo del usuario	Sí: la sección declara la limitación
Automatización (System Events)	Solo al pulsar «Consultar login items»	Consultar la lista de login items	Sí: la lista queda vacía
Root	Nunca se pide	—	—

El producto **no ejecuta sudo en ninguna parte**. Cuando un dato requiere privilegios que no tiene, usa una vía alternativa (SSH deducido de `launchctl list` en vez de `systemsetup`) o declara la limitación.

Es coherente con el `Info.plist`, que solo declara `NSAppleEventsUsageDescription` y añade un comentario explicando que el producto no accede a cámara, micrófono, contactos ni ubicación.

12. Autoprotección del agente

Control	Qué detecta	Qué no hace
Latido	Que la sesión sigue viva	No reinicia el agente
Cierre abrupto	Que la sesión anterior no cerró limpiamente	No impide que ocurra
Ventana de reinicios	Cierres inesperados repetidos	No bloquea nada
Huella de configuración	Que el archivo cambió entre sesiones	No detecta un cambio malicioso indistinguible en tamaño y fecha

La huella es tamaño + fecha de modificación, **no un hash**, y el código lo declara. Un atacante que edite la configuración preservando ambos —posible con `touch`— pasaría inadvertido. Registrado como riesgo.

Además, el producto **no se protege contra su propia finalización**: cualquiera que pueda enviar señales al proceso puede detenerlo. Lo que hace es dejar constancia en el arranque siguiente.

13. Controles implementados — resumen

Control	Estado
Sin ejecución de shell	■
Rutas absolutas a todas las utilidades del sistema	■
Parámetros ligados en SQL	■ (con la excepción documentada de identificadores)
Apertura de bases ajenas en solo lectura	■
Sin escalada de privilegios	■
Secretos fuera de los archivos de configuración	■
Ninguna acción automática	■
Lista de procesos protegidos	■
Confirmación de dos pasos para borrar	■
Auditoría de acciones	■
Sin telemetría	■
Salida a red apagada por defecto	■
Verificación de certificados TLS	■ (por defecto de <code>curl</code>)
Hashes de integridad en las releases	■
<code>clippy -D warnings</code> en CI	■

14. Controles ausentes o no comprobados

Control	Estado	Impacto
Firma y notarización de los binarios	Ausente, declarado en el README	El usuario debe autorizar la app a mano o compilarla
cargo audit en la CI	Ausente	Vulnerabilidades conocidas de dependencias sin vigilar
Fijación por SHA de las acciones de CI	Ausente	Riesgo bajo de cadena de suministro
Versión fijada de markdownlint-cli2	Ausente	Idem
Comprobación de que ai.endpoint sea HTTPS	Ausente	Un endpoint en claro expondría incidente y clave
Clave de IA fuera de los argumentos de curl	Ausente	Visible en ps durante la petición
Hash criptográfico de la configuración	Ausente por diseño declarado	Cambio malicioso indetectable si preserva tamaño y fecha
Protección de la baseline contra manipulación	Ausente	Quien escriba en el SQLite puede blanquear un hallazgo
Permisos restrictivos en los archivos creados	No comprobado	Dependen de la umask
Saneado completo del AppleScript de notificación	Parcial	Explotabilidad baja: el texto lo genera el producto
Anonimización de reportes y exportes	Ausente	El usuario debe revisarlos antes de compartir
Análisis SAST específico más allá de clippy	Ausente	—

15. Qué se probó y qué no en este análisis

Comprobación	Resultado
Búsqueda de secretos literales en el repositorio	Sin hallazgos
Búsqueda de ejecución de shell (sh -c, Command::new("sh"))	Sin hallazgos
Revisión de todas las consultas SQL	Parámetros ligados salvo identificadores documentados
Revisión de todas las llamadas a Command	Todas con ruta absoluta y argumentos separados
Revisión de la salida de red	Una sola, condicionada por tres guardas
cargo clippy --all -targets --all-features -- -D warnings	Sin advertencias
cargo test --all-features	112 pruebas en verde
cargo audit	No ejecutado
Pruebas dinámicas o de intrusión	No realizadas: quedan fuera del alcance de una documentación

Comportamiento con <code>TCC.db</code> legible	No verificado: el terminal de este análisis no tiene Acceso total al disco
--	--

Siguiente lectura recomendada: [12 · Pruebas y calidad](#).